

# CSP-J 初赛知识点讲义：进制转换 · 原反补 · 位运算 · 逻辑运算与 lowbit

适用：CSP-J（入门级）第一轮认证（初赛，笔试/机考选择题）

建议用时：讲义通读 1.5 小时 + 配套练习题 1 小时

## 一、进制转换

### 1.1 常见进制与符号

| 进制   | 英文名     | 数字符号            | 后缀/前缀  |
|------|---------|-----------------|--------|
| 二进制  | Binary  | 0,1             | B / 0b |
| 八进制  | Octal   | 0-7             | o / 0  |
| 十进制  | Decimal | 0-9             | D / 无  |
| 十六进制 | Hex     | 0-9, A-F(10-15) | H / 0x |

字母大小写均可，A=10, B=11, C=12, D=13, E=14, F=15。

### 1.2 其它进制 → 十进制（按权展开）

公式：（各位数字 × 基数的位权）之和，位权从右往左为 基数<sup>0</sup>，基数<sup>1</sup>，...

- 1  $(1011)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8+0+2+1 = 11$
- 2  $(65)_8 = 6 \times 8^1 + 5 \times 8^0 = 48+5 = 53$
- 3  $(1A)_{16} = 1 \times 16^1 + 10 \times 16^0 = 16+10 = 26$

## 1.3 十进制 → 其它进制

**整数部分：除基取余，逆序排列**（除到商为 0，余数从下往上读）

把 13 转二进制：

- |   |                 |     |             |
|---|-----------------|-----|-------------|
| 1 | $13 \div 2 = 6$ | 余 1 | ↑           |
| 2 | $6 \div 2 = 3$  | 余 0 |             |
| 3 | $3 \div 2 = 1$  | 余 1 |             |
| 4 | $1 \div 2 = 0$  | 余 1 | ↓ （读序：从下往上） |

→  $(1101)_2$

**小数部分：乘基取整，顺序排列**（CSP-J 初赛基本不考小数，了解即可）

## 1.4 二进制 ↔ 八进制 ↔ 十六进制（成组互换）

- **二 → 八**：从右往左每 **3 位** 一组，不足补 0，每组转 1 位八进制。
- **二 → 十六**：从右往左每 **4 位** 一组，不足补 0，每组转 1 位十六进制。
- 反向同理（一位拆成 3 位 / 4 位）。

- 1  $(110101)_2 \rightarrow 110 \mid 101 \rightarrow (65)_8$
- 2  $(11010110)_2 \rightarrow 1101 \mid 0110 \rightarrow (D6)_{16}$
- 3  $(D6)_{16} \rightarrow 1101 \ 0110 \rightarrow (11010110)_2$

## 1.5 必须记住的速算

- $2^{10} = 1024$ ,  $1\text{KB} = 1024\text{B}$ ,  $1\text{MB} = 1024\text{KB}$
- $16 = 2^4$ : 1 位十六进制 = 4 位二进制
- $8 = 2^3$ : 1 位八进制 = 3 位二进制
- 权值表:  $2^0=1, 2^1=2, 2^2=4, 2^3=8, 2^4=16, 2^5=32, 2^6=64, 2^7=128, 2^8=256$

## 1.6 初赛常考形式

1. 直接在二/八/十/十六之间互转填空。
2. 给一段二进制, 问「包含几个 1」「最高位权值是多少」。
3. 问  $n$  位二进制能表示多少个数 ( $=2^n$ )、能表示的最大十进制数 ( $=2^n-1$ )。

# 二、原码、反码、补码

计算机内部用**补码**存储整数。这三者只在「有符号整数」「负数表示」上有区别。

## 2.1 机器数的结构 (以 $n$ 位为例)

- **最高位 = 符号位**: 0 表示正, 1 表示负。
- 其余  $n-1$  位 = 数值位 (真值的二进制)。

## 2.2 原码

- 正数: 符号位 0 + 真值二进制。
- 负数: 符号位 1 + 真值二进制。
- 例 (8 位):  $+5 \rightarrow 0000\ 0101$ ;  $-5 \rightarrow 1000\ 0101$

缺点: 0 有两种表示 ( $0000\ 0000$  与  $1000\ 0000$ ), 且加减要分正负判断。

## 2.3 反码

- 正数：与原码相同。
- 负数：符号位不变，数值位按位取反。
- 例（8 位）：-5 反码 = 1111 1010

## 2.4 补码

- 正数：与原码相同。
- 负数：反码 + 1。
- 例（8 位）：-5 补码 = 反码 1111 1010 + 1 = 1111 1011

## 2.5 为什么计算机用补码

1. 减法变加法：  $a - b = a + (\text{补码表示的 } -b)$ ，一套加法电路搞定加减。
2. 0 只有一种表示：0000 0000，没有「±0」歧义。
3. 表示范围比原码多一个数。

## 2.6 表示范围（n 位有符号整数，补码）

|   |                             |                       |
|---|-----------------------------|-----------------------|
| 1 | 最小值 = $-2^{(n-1)}$          | 最大值 = $2^{(n-1)} - 1$ |
| 2 | 8 位： [-128, 127]            |                       |
| 3 | 16 位： [-32768, 32767]       |                       |
| 4 | 32 位： $[-2^{31}, 2^{31}-1]$ |                       |

易错：-128 没有对应的原码和反码，只有补码 1000 0000。这是补码比原码多表示一个负数的原因。

## 2.7 由补码求真值（两步法）

- 符号位为 0 → 正数，直接按权展开。
- 符号位为 1 → 负数，对补码再求一次补（取反 + 1）得到绝对值，再加负号。

|   |                                     |
|---|-------------------------------------|
| 1 | 补码 1111 1011                        |
| 2 | 符号位 1 → 负数                          |
| 3 | 取反+1: 0000 0100 + 1 = 0000 0101 = 5 |
| 4 | 真值 = -5                             |

## 三、位运算（Bitwise）

C++ 中位运算直接操作整数的二进制位，是初赛和复赛的高频考点。

### 3.1 六种运算符

| 运算符 | 名称   | 规则                               |
|-----|------|----------------------------------|
| &   | 按位与  | 对应位 <b>都为 1</b> 才得 1             |
|     | 按位或  | 对应位 <b>有 1</b> 即得 1              |
| ^   | 按位异或 | 对应位 <b>不同</b> 为 1， <b>相同</b> 为 0 |
| ~   | 按位取反 | 0 变 1，1 变 0（包括符号位）               |
| <<  | 左移   | 全体左移 k 位，低位补 0（等价 $\times 2^k$ ） |
| >>  | 右移   | 全体右移 k 位（等价 $\div 2^k$ 向下取整）     |

|   |                                            |
|---|--------------------------------------------|
| 1 | 5 & 3 : 0101 & 0011 = 0001 = 1             |
| 2 | 5   3 : 0101   0011 = 0111 = 7             |
| 3 | 5 ^ 3 : 0101 ^ 0011 = 0110 = 6             |
| 4 | ~5(8位): 0000 0101 → 1111 1010 (= 补码表示的 -6) |
| 5 | 5 << 1 : 0101 → 1010 = 10 (×2)             |
| 6 | 6 >> 1 : 0110 → 0011 = 3 (÷2)              |

## 3.2 异或 (^) 的重要性质

- $a \wedge a = 0$
- $a \wedge 0 = a$
- 交换律:  $a \wedge b = b \wedge a$
- 结合律:  $(a \wedge b) \wedge c = a \wedge (b \wedge c)$
- 应用: **不用临时变量交换两个数**

```
1 a ^= b;    // a = a^b
2 b ^= a;    // b = b^(a^b) = a
3 a ^= b;    // a = (a^b)^a = b
```

## 3.3 取位 / 置位 / 清位 / 翻位的模板

设  $k$  从 0 开始编号 (最右是第 0 位) :

```
1 取第 k 位      : (x >> k) & 1
2 把第 k 位置 1  : x | (1 << k)
3 把第 k 位置 0  : x & ~(1 << k)
4 把第 k 位翻转  : x ^ (1 << k)
5 取低 n 位      : x & ((1 << n) - 1)
6 判断奇偶       : (x & 1) == 1 → 奇数
```

## 3.4 运算符优先级 (初赛必考易混)

从高到低 (节选中与本题相关的) :

```
1 ~ (按位取反)
2 算术: * / % + -
3 移位: << >>
4 关系: < > <= >= == !=
5 按位: & 高于 ^ 高于 |
6 逻辑: && 高于 ||
7 赋值: = += ...
```

关键易错点：位运算整体低于算术和关系运算；`&` 高于 `^`  
高于 `|`；`&&` 高于 `||`。  
例：`3 + 5 & 1` 先算 `3+5=8`，再 `8 & 1 = 0`。

## 四、逻辑运算

### 4.1 三种逻辑运算符

| 运算符                     | 名称  | 结果            |
|-------------------------|-----|---------------|
| <code>&amp;&amp;</code> | 逻辑与 | 两边都「真(非0)」才为真 |
| <code>  </code>         | 逻辑或 | 有一边为真即为真      |
| <code>!</code>          | 逻辑非 | 取反            |

- 操作数是「真假」概念：`0` 为假，非 `0` 为真。
- 结果类型是 `bool`，在 C++ 中 `true` 当作 `1`、`false` 当作 `0`。

### 4.2 短路求值（重点中的重点）

- `a && b`：若 `a` 已经为假，`b` 不再计算。
- `a || b`：若 `a` 已经为真，`b` 不再计算。

```
1 int x = 0;  
2 if (x != 0 && 10 / x > 2) { }    // 安全：x==0 时右边  
    不执行，不会除零
```

初赛常出题：「以下代码是否会崩溃 / `b` 是否会执行」  
——核心就是看左边是否已能决定结果。

## 4.3 逻辑运算 vs 位运算（经典易错对）

```
1 1 && 2 = 1 (两边都非0 → 真)
2 1 & 2 = 0 (二进制 01 & 10 = 00)
3 1 || 2 = 1
4 1 | 2 = 3 (01 | 10 = 11)
5 !5 = 0
6 ~5 = ... (按位取反, 得到补码形式的负数)
```

记住: `&&` `||` 看「真假」, 返回 0/1; `&` `|` 看「每一位」, 返回按位结果。

## 五、lowbit

### 5.1 定义

```
1 lowbit(x) = x & (-x)
```

含义: 取出 `x` 的二进制表示中, **最低位的一个 1 及其后面所有 0** 所组成的数值。

### 5.2 原理（为什么 `x & (-x)` 能取到最低位 1）

`-x` 在计算机里是 `x` 的**补码**（按位取反 + 1）。取反后, 最低位 1 右边本来全是 0 → 取反后全变 1; 左边全部取反。再 +1 会把这一串低位的 1 进位, 恰好只保留住原来的最低位 1, 其余位变 0。于是 `x & (-x)` 仅该位为 1。

```
1 x      = 12 = 0000 1100
2 ~x      = 1111 0011
3 -x = ~x+1 = 1111 0100 (= -12 的补码)
4 lowbit(12) = 1100 & 0100 = 0100 = 4
```



## 5.3 常见应用

1. 统计二进制中 1 的个数：不断 `x -= lowbit(x)` (或 `x &= x-1`)，计数到 0。

```
1 int cnt = 0;
2 while (x) { x -= lowbit(x); cnt++; } // 或
   x &= (x-1);
```

2. 判断 `x` 是否为 2 的幂： `(x & (-x)) == x` (且 `x != 0`)。

## 5.4 一组配套位技巧

```
1 lowbit(x)      = x & (-x)      // 取最低位 1 的值
2 x & (x - 1)     = 消去最低位的 1
3 x ^ (x - 1)     = 最低位 1 及以下全变 1
4 x & (~ (x - 1)) = 等价于 lowbit(x)
5 __builtin_popcount(x) // C++ 内置：返回 x 中 1 的个数
```

# 六、初赛高频易错清单（考前再过一遍）

1. `&` 与 `&&`、`|` 与 `||` 的区别（真假 vs 按位）。
2. 负数用补码存储；-128 没有原码/反码。
3. 补码求真值：符号位 1 时「再求一次补」。
4. `<< k` =  $\times 2^k$ ，`>> k` =  $\div 2^k$ （向下取整）。
5. 逻辑短路：左边已能定结果，右边不执行。
6. 位运算优先级低于算术/关系；`&` > `^` > `|`；`&&` > `||`。
7. `lowbit(x) = x & (-x)`，可判断 2 的幂、数 1 的个数。

